M3 SOLO PROJECT

ItemVault Design Report

A complete explanation of the user interface, visual design, navigation, features, components, authentication, and data choices used in the personal inventory app.

Product	Stack	Date
ItemVault	React, Vite, Firebase	June 5, 2026

Project Overview

ItemVault is a personal inventory manager designed to help a user record, organize, search, and update belongings in one clean dashboard. The design focuses on clarity: users can quickly see what they own, how much their owned items are worth, where an item is located, and whether an item is owned, sold, lost, or borrowed.

> **Main design idea:** the app should feel simple, trustworthy, and easy to scan, so the user can manage inventory details without feeling like they are using a complicated admin system.

Design Goals

Simple Organization

The dashboard brings search, category filters, status filters, stats, and item cards together so the user can find inventory information quickly.

Secure Personal Data

Authentication protects product pages, while Firestore stores each user's items under that user's account.

Consistent Experience

Cards, rounded controls, color tokens, spacing, and repeated page structures keep the app familiar from screen to screen.

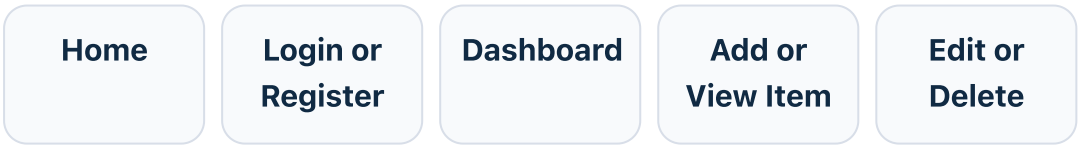
Visual System

The design uses a calm light theme with strong indigo actions, white surfaces, soft borders, and rounded cards. A dark theme is also available from the navigation bar and is stored in `localStorage` so the user's preference remains after refresh.

- **Typography:** system sans-serif fonts are used for a modern, readable interface.
- **Shape language:** large rounded cards and pill buttons create a friendly visual style.
- **Elevation:** shadows separate important surfaces like cards, forms, and dashboard panels.
- **Status color:** item status tags use green, red, amber, and blue to make state easy to scan.

User Flow

The app is organized around a short, practical journey: users enter through the home page, sign in or register, then manage their inventory from the protected dashboard.



Screen-by-Screen Design

Home Page

The home screen introduces ItemVault with a clear headline, short explanation, and two primary actions: view the dashboard or add an item. A supporting panel explains the product value in plain language.

Login and Register

Authentication screens use centered cards, clear labels, error messages, and Google sign-in buttons. Register also captures first name and surname so the app can greet the user personally.

Dashboard

The dashboard welcomes the user, shows total items and owned value, then provides search, category filtering, and status filtering before listing matching item cards.

Add Item

The add form collects the main inventory details: name, category, price, condition, status, location, purchase date, image URL, and optional notes.

Item Details

Each item has a focused detail page with image support, item metadata cards, and actions to edit or delete the item.

Edit Item

Editing reuses the same product form with existing item data pre-filled, keeping the create and update experiences consistent.

Navigation and Protection

Routing is handled through React Router. Public pages include Home, Login, and Register. Inventory pages are grouped inside a protected route area, so dashboard, add item, item details, and edit item screens only render for authenticated users.

- `/` shows the public home page.
- `/login` and `/register` handle account access.
- `/dashboard`, `/add-item`, `/items/:id`, and `/items/:id/edit` are protected inventory routes.
- The navbar changes based on login state, showing dashboard, add item, greeting, and logout only for signed-in users.

Reusable Components

ProductForm

Reused for both adding and editing items. It validates required fields, prevents negative prices, and disables submission until the form is valid.

ProductCard

Presents item image, category, status, notes, location, price, purchase date, and a view action in a scannable card layout.

SearchBar and FilterDropdown

Keep dashboard controls reusable and accessible with labels and controlled input values.

InventoryStats

Summarizes total item count and total owned value so the dashboard gives an immediate overview before the item grid.

Data and Authentication Design

The app uses Firebase Authentication for account management and Firestore for inventory storage. The product context listens to the current user's Firestore items in real time, filters by `userId`, and orders the results by creation date.

- **Email/password authentication:** users can create an account and log in with credentials.
- **Google authentication:** users can continue with a Google account.
- **User-scoped data:** each item is saved with the current Firebase user's ID.
- **Real-time updates:** Firestore snapshots keep the dashboard synced with database changes.

Interaction Details

Search and Filters

The dashboard uses memoized filtering to combine item name search, category selection, and status selection. This keeps the interface responsive and predictable.

Validation

The product form validates item name, category, price, condition, location, purchase date, and status. Inline errors help users fix issues before saving.

Empty States

The product list gives different messages when the inventory is empty versus when filters return no results, making the next step clear.

Personalization

The dashboard and navbar greet the user by first name or email prefix, making the product feel personal without adding complexity.

Responsive Design

The layout uses CSS Grid, flexible widths, and media queries to adapt from desktop to smaller screens. The hero, detail grid, dashboard controls, and form rows collapse into one-column layouts on narrow screens so content remains readable and controls stay easy to tap.

Accessibility Choices

- Inputs use visible labels or accessible labels.
- Product images include alt text based on the item name.
- Buttons include disabled states while forms are submitting.
- Color is supported by readable text labels, so status is not communicated by color alone.
- Loading and not-found states explain what is happening instead of leaving blank screens.

Implementation Files

The main design and behavior are split across these project files:

- `src/App.jsx` - route structure and lazy-loaded pages
- `src/App.css` - visual system, layout, cards, forms, and responsive rules
- `src/pages/Home.jsx` - landing experience
- `src/pages/Products.jsx` - dashboard, filtering, and stats
- `src/pages/AddProduct.jsx` - create item page
- `src/pages/ProductDetails.jsx` - item detail and delete action
- `src/pages/EditRequest.jsx` - edit item page
- `src/components/ProductForm.jsx` - reusable add/edit form
- `src/components/ProductCard.jsx` - reusable item card
- `src/context/AuthContext.jsx` - authentication state and actions
- `src/context/ProductContext.jsx` - Firestore inventory data
- `src/firebase/firebaseClient.js` - Firebase setup

Final Note

The current implementation stores live inventory data with Firebase/Firestore. Some interface copy still mentions `localStorage`, which looks like earlier project wording. For the final submission, that copy should be updated to say Firebase/Firestore if the cloud data design is the intended final design.

Generated for the ItemVault M3 Solo Project. Source: `docs/itemvault-design-report.html`.